StudyQuest User Documentation

SDEV265, Fall 2026

Team 3

Table of Contents

Introduction

This document is intended to provide instructions to those using the Third Team's StudyQuest application.

StudyQuest is a lightweight desktop application that helps students and learners organize one-time tasks and recurring study habits and motivates consistent work with a simple gamification mechanic: experience points (XP) and levels. The program provides an easy graphical interface (Tkinter) allowing users to add tasks with optional due dates, create habits with daily/weekly frequency, mark goals complete, and see progress visually. Data is persisted to a human-readable save file so the user's profile and goals are automatically restored at the next launch.

This documentation explains installation, operation, mapping of requirements to implementation, and troubleshooting.

Installation and Requirements

System requirements are as follows:

- Have Python 3.8 and higher (recommended 3.9/3.10/3.11) installed and on PATH.
- No external packages are required. The GUI uses the standard tkinter library that ships with most Python distributions on Windows and macOS. Linux users may need to install python3-tk or equivalent package.

Files:

- StudyQuest_v3.py – is a single Python file for the complete application.
- studyquest_save.txt – is auto-created in the same folder as studyquest.py when you save for the first time.
- studyquest_save.txt.bak – is automatically created when trying to delete the profile. In case the deletion fails, the progress is restored from this file. In case the deletion succeeds, this file is deleted.

Installation steps:

1) Download the latest version of the StudyQuest Program (currently v3)
2) Place StudyQuest_v3.py in a folder you control.
3) Open a terminal / command prompt and cd to that folder.
4) Run: python StudyQuest_v3.py
5) If a save file exists in the folder, StudyQuest will load it; otherwise, the app prompts your name and creates a new profile.

Getting Started:
First Start of the Program and Player Profile Creation

Follow these steps to register players for a new game:

1) Launch the program as described above.
2) On first launch (no studyquest_save.txt present) a small dialog appears asking for your name. Type your name and press OK. This creates a profile for the player. In case no name is provided, or Cancel button, or close button is pressed, then the program will use the name “Student”.
3) The main window shows your profile (name, level, XP), simple statistics, and a table of goals (Task or Habit). Use the buttons on the top-right to add a task/habit, complete a selected goal, or save manually. The app also saves automatically when the window is closed to prevent potential unintended progress loss.

Main Features of StudyQuest

- Profile & Progress: name, level, current XP and XP required for next level are displayed at the top.

- Add Task button: one can add a one-time task with title, description, difficulty (Easy/Medium/Hard) and optional due date in the format of YYYY-MM-DD. Please note that tasks may be completed only once, but after completed, they also may be deleted using the "delete selected" button.

- Add Habit button: one can add a recurring habit with title, description, difficulty and frequency (Daily/Weekly). Habits track current streak, best streak, and last completion date. They can be logged repeatedly.

- Filter: when looking for a specific goal, one may leverage the filter options: show tasks, habits, and completed. One may also use the search bar to look up something specific. One may look up a goal by typing in goal's title, description, difficulty, status and streak.

- Complete Selected button: choose a row in the table and click "Complete Selected" to mark a task done or log a habit. XP is awarded based on difficulty and streak bonus for habits. Level ups are handled automatically and displayed.

- Delete Selected button: if one wishes to delete a task or a habit, they may utilize the "Delete Selected" button. The goal will disappear. This does not hurt one's XP.

- Delete Profile button: In case one would like to start from scratch, they may use the "Delete Profile" button that is red-colored. One will need to confirm their decision, and after the deletion, the program will close out. Launch the program again to start fresh.

- Security feature: when deleting a profile, studyquest_save.txt.bak is automatically created. In case the deletion fails, the progress is restored from this file. In case the deletion succeeds, this file is deleted.

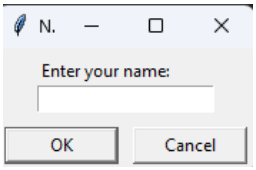
- Saving the progress: one may use the Save button or close the window to save current profile and goals to studyquest_save.txt. The app also saves automatically on close to prevent potential unintended progress loss. The app loads from that save file on startup if present, and if not, it will ask for a name again to set up a profile.

Implementation Details

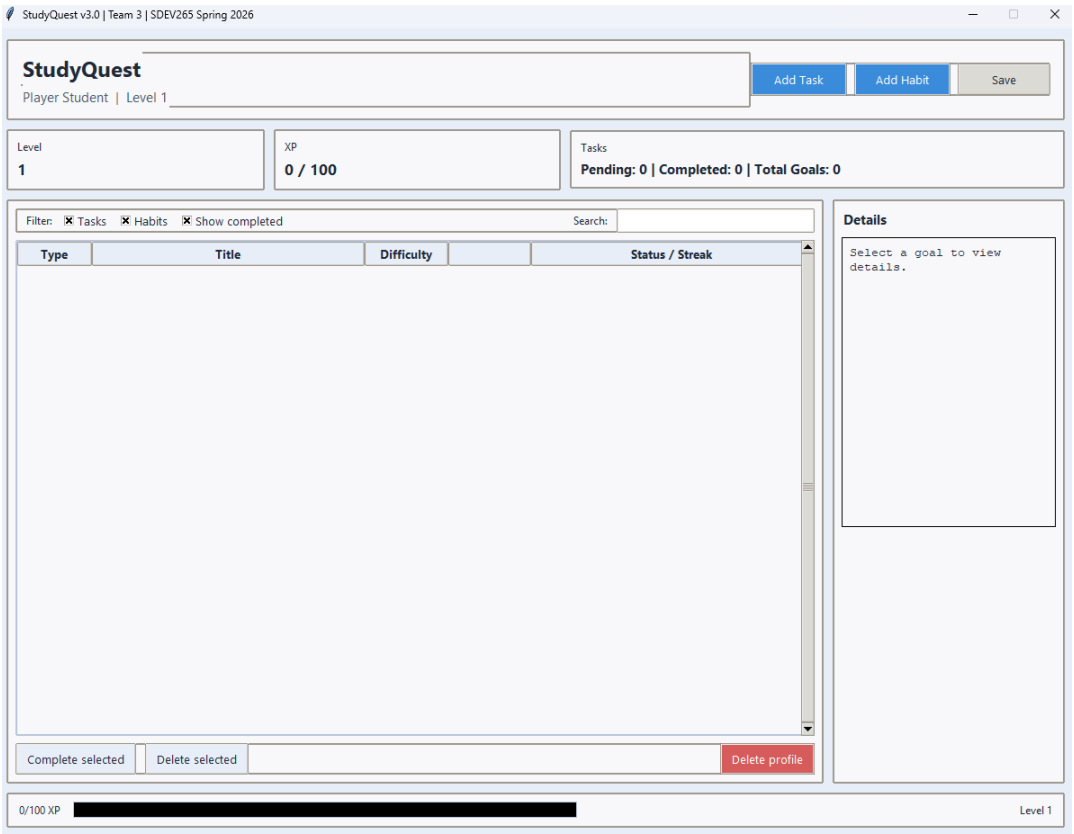
Create and Load Profile

Requirement: The program must allow creating a new user profile and persist the name, so the profile is automatically restored on subsequent launches.

Implementation: At startup the backend Storage class checks for studyquest_save.txt. If the file exists and is valid, Storage.load() reconstructs the Player object from the PLAYER|... line. If the file does not exist or fails to parse, the front end prompts the user via a simpledialog.askstring dialog for their name. The Player dataclass stores name, level, current_xp, and xp_to_next. This flow guarantees that the user supplies a name only when needed and that saved profiles are reused automatically.



Picture 1. Entering the name.

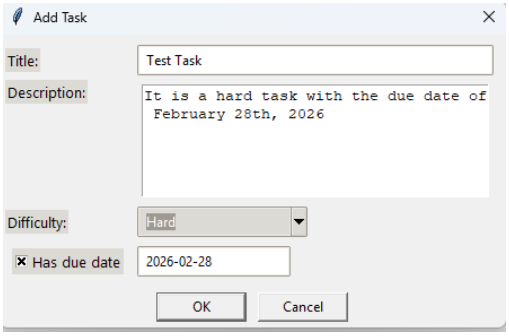


Picture 2. Main Window after Profile Creation.

Add Task, a one-time goal

Requirement: Users should be able to add a one-time task with title, description, difficulty, and optional due date.

Implementation: The frontend exposes AddTaskDialog which collects Title, Description, Difficulty (combobox: Easy/Medium/Hard), and optional Due Date (user toggles "Has due date"). When the dialog is accepted, the frontend calls GoalManager.add_task(title, desc, difficulty, due_date) to create a Task object. The Task class stores due_date and completed status and implements complete() to award base XP plus a due-date bonus if completed on or before the due date.

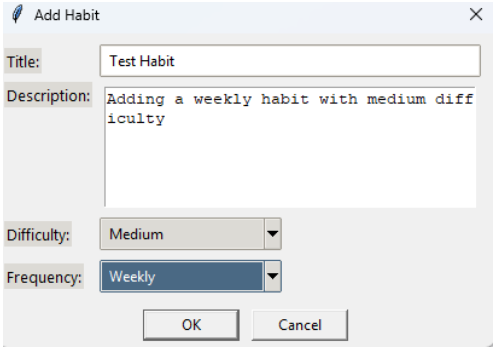


Picture 3. Adding a task.

Add Habit, a recurring goal

Requirement: Users should be able to add recurring habits with frequency and track streaks.

Implementation: The AddHabitDialog collects Title, Description, Difficulty, and Frequency (Daily/Weekly). The frontend calls GoalManager.add_habit(...). The Habit class tracks current_streak, best_streak, and last_completed date. Its complete(when) method compares the new date with the last completion date and updates the streak according to frequency rules (daily must be consecutive days; weekly allows up to 7-day gaps). The method returns XP computed from base difficulty plus a small streak bonus, encouraging consistent habit completion.

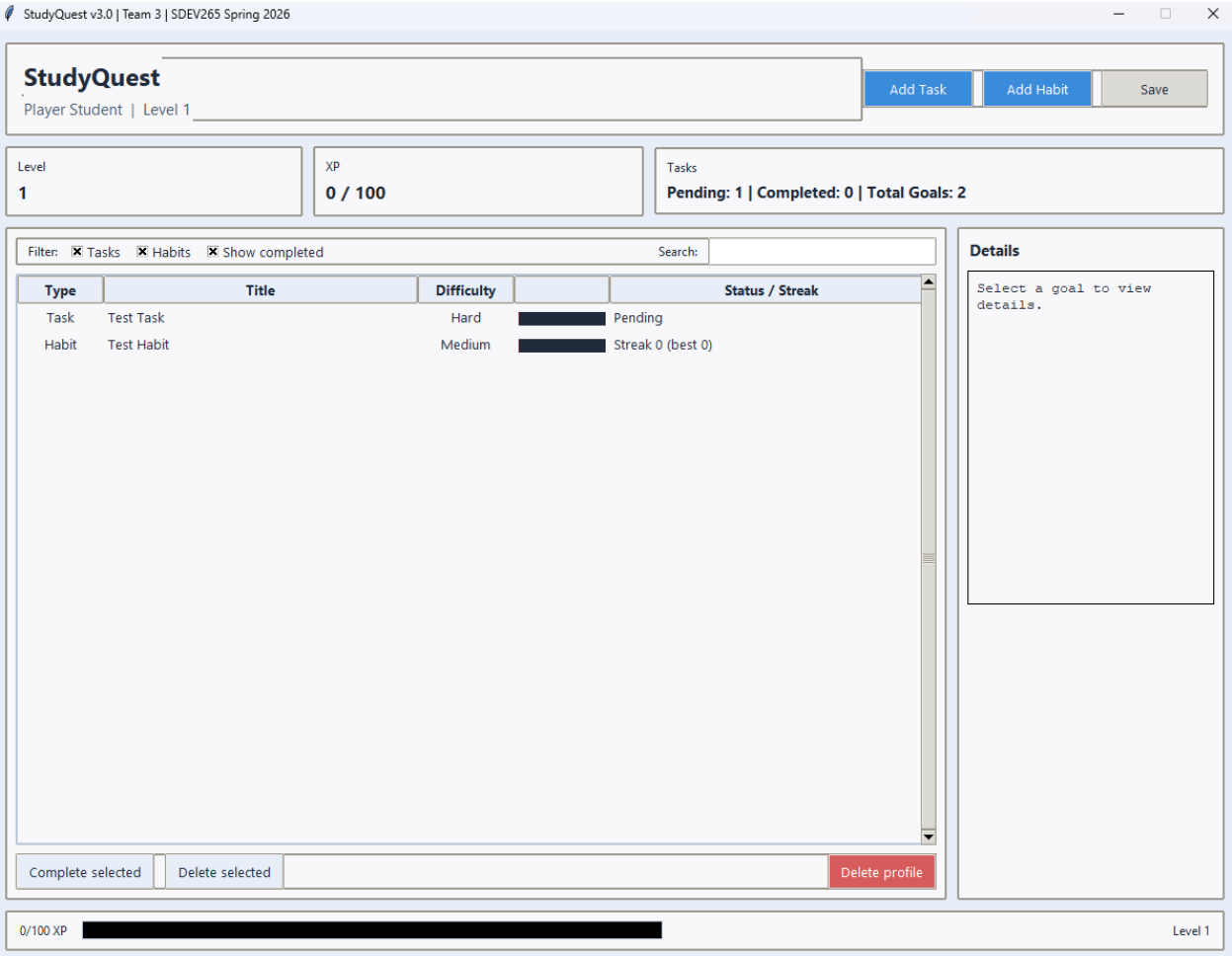


Picture 4. Adding a habit.

Complete Goal, XP and Leveling

Requirement: Completing tasks/habits should award XP and automatically manage leveling.

Implementation: The backend defines Player.add_xp(amount) which increases current_xp and checks thresholds to increment level and increase xp_to_next. When the GUI triggers GoalManager.complete_by_id(id, when), the corresponding goal's complete() logic returns the XP to award. The frontend then applies player += xp, displays messages for XP gained and level ups, and refreshes UI, ensuring consistent business logic lives exclusively in backend classes.

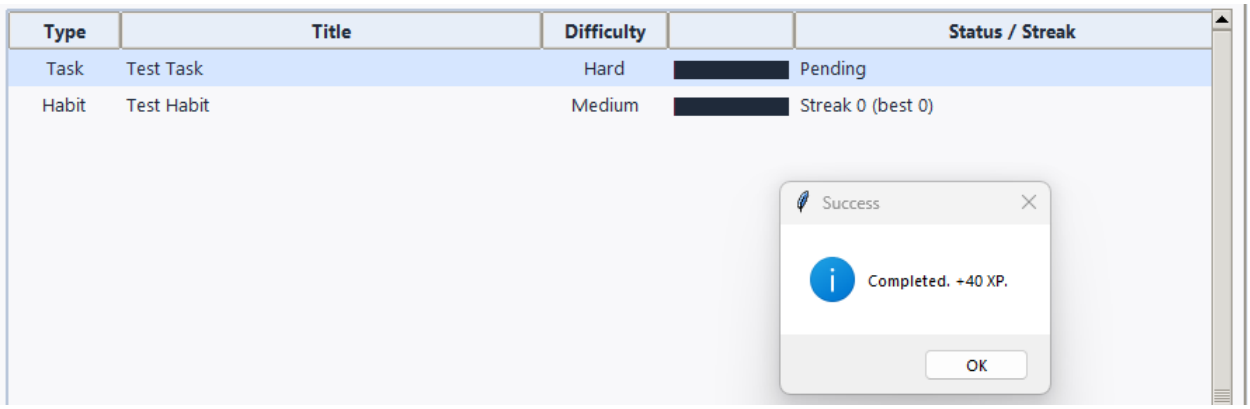


Picture 5. Several goals are added.

Save and Load Profile Progress

Requirement: The program must save user profile and goals to a file and load them later, handling corrupted or missing files gracefully.

Implementation: Storage.save(path, player, goal_manager) writes a temporary file then replaces the save file to avoid corruption. It writes a single PLAYER|... line followed by GOAL|TASK|... or GOAL|HABIT|... lines. Storage.load() parses these lines, constructs Player, Task, and Habit objects, and recalculates next IDs. Error handling wraps parsing to present a clear message and fallback to prompting for name when necessary.



Picture 6. Completing a goal.

Robustness and Error Handling

Requirement: The app should validate user inputs (e.g., non-empty titles, valid date formats) and not crash on malformed files.

Implementation: validate() methods for dialogs enforce non-empty titles and correct YYYY-MM-DD date format when users choose a due date. Storage functions use exception handling to surface a friendly warning box and to create a fresh profile if loading fails. GUI message boxes show helpful errors for invalid operations. For example, attempting to complete an already-completed task returns 0 XP and informs the user.

Try It Yourself: Step-by-Step Program Usage Example

First, start the app and create a profile (or the program will load it for you if one already exists):

1. Run python studyquest.py. If a valid studyquest_save.txt exists, it will load automatically.
2. If no save file exists, a dialog asks “Enter your name:” – type your name and press OK.

Second, add a one-time task:

1. Click Add Task.
2. Fill Title (required), Description (optional), select Difficulty, toggle “Has due date” if needed and set date according to the following format: YYYY-MM-DD.
3. Click OK. The task appears on the table.

Third, add a habit:

1. Click Add Habit.
2. Fill Title, Description, select Difficulty and Frequency (Daily or Weekly).
3. Click OK. The habit appears with initial streak 0.

Then, complete a goal:

1. Complete a goal in real life.
2. Select the row corresponding to the goal in the list.
3. Click Complete Selected button.
4. A pop-up indicates XP gained and if the user leveled up. The UI updates to show new XP, level, and updated streaks/status.

Finally, save progress:

- ✓ Click Save to save the progress manually or simply close out the window since the app saves the progress automatically to prevent potential unintended progress loss. The file studyquest_save.txt will be written into the same folder.
- ✓ It will also be read from on the next launch of the program in order to load the progress.

Troubleshooting and FAQ

Q: App won't launch because "tkinter not found"

A: Install the Tkinter package. On Linux: sudo apt install python3-tk. On Windows/macOS, use a standard Python installer (official distributions include tkinter). You may need to reinstall Python. Follow the recommended installation instructions instead of customizing when installing Python fresh.

Q: Save file seems corrupted and app fails to load

A: The app displays a warning and will prompt a name to start fresh. You can also open studyquest_save.txt in a text editor to inspect lines and manually remove bad lines. If needed, simply delete the save file and try starting fresh.

Q: Date format errors when adding a task

A: The due date must be the following format: YYYY-MM-DD. For example, 2026-02-28 will work, but 2/28/2026 will not. The Add Task dialog validates this format and will show a warning if invalid.

Q: Habit completed twice in same day gives XP again

A: The program prevents duplicate habit logging for the same day. If tried, the program returns 0 XP and the GUI shows "No XP gained…" if tried to complete a habit twice on the same date. The program knows what date it is today and checks it to make sure habits are not logged multiple times.

Summary

The StudyQuest application is a single-user desktop program designed to help individuals manage tasks and build consistent study habits through a gamified XP and leveling system. If a studyquest_save.txt file exists in the program folder, the application automatically loads the saved profile; if not, the user is prompted to enter their name when launching the program for the first time. It was developed in Python using the Tkinter library and runs locally in a desktop environment.

All core features outlined in the project requirements have been implemented, including adding one-time tasks with optional due dates, creating recurring daily or weekly habits with streak tracking, awarding XP based on difficulty, automatic level progression, and reliable file-based saving and loading.

Although there is room for expansion, StudyQuest in its current state provides a stable, functional and user-friendly tool for personal productivity and habit building. With additional refinements and expanded features, it could evolve into a more comprehensive study management system while maintaining its clear and accessible design.